

Team Engine Documentation

CSC 481/581 - Game Engine Foundations - Fall 2026

Group 4: Adit Jigneshbhai Shah, Sai Sumedh Kaveti, Yeva Mheryan

Milestone 2 - Integrate Conceptual Models to the Game Engine Project 2 - Time and Networking Foundations

GitHub Repository: <https://github.com/CodeRockerr/CSC481-581-Group4>

(A) Introduction

The addition of time and networking in Milestone 2 extends our existing common engine with time modeling and game loop creation on worker threads, as well as the world being shared across multiple windows either by headless server or peer-to-peer with no authority. As is the case with Milestone 1, these new additions reside in our engine library, with each individual's game being able to utilize them as desired.

(B) Team Contributions

Member	Branches / PRs	Engine work
Adit Jigneshbhai Shah	feat/net-message, feat/p2p, fixes in feat/server	NetMessage packet reuse, handshake on the server side and client disconnect fixes, windowed demo client, shared peer clock, peer-to-peer with no server, pacing based on the player timeline (Engine::paceFrame), additional design information in Docs/Design
Sai Sumedh Kaveti	feat/timeline, feat/client-threads	Global and local timelines with tick size, anchoring, scale, and pause; entity-based timelines (WorldTime, PlayerTime); multi-threaded client loop with 2 frame workers and entity-based mutexes
Yeva Mheryan	feat/server	ZeroMQ dependencies, headless client-server networking: Single-threaded server with threads per client, NetworkClient having threads for

		receiving data
--	--	----------------

Each member also built an individual Milestone 2 game on top of the engine: Cave Ninja (Sai), Lost Under the Sea (Adit) and Hello Kitty Adventure (Yeva).

(C) Milestone Requirement Mapping

Section	Requirement	Implementation
1 - Time	Timelines: tic, anchor, scale, pause	Timeline.cpp, Engine.cpp
2 - Client-Server	Headless server, 3+ clients, shared world	NetMessage.h, Server.cpp, NetworkClient.cpp
3 - Multithreading	Frame split across threads	FrameWorker.cpp, Entity::stateMutex
4 - Asynchronicity	Clients at different speeds	Engine::paceFrame, one server thread per client
5 - Peer-to-peer	Option 2: no central authority	PeerClock.h, PeerSession.cpp

(D) Section 1 - Timelines

The global timeline calls `SDL_GetTicksNS` and is never paused or scaled. Local timelines are linked to the global timeline or another local timeline, having tic size (1 ms default), scale, and whether it is paused. On each step, the local timeline adds $(\text{now} - \text{anchorLast}) * \text{scale}$ to its own time if it is not paused. `WorldTime` governs the world, and `PlayerTime` governs the player: P pauses it, while 1 / 2 / 3 change its scale to 0.5, 1.0, and 2.0. The entities store the `timelineId` and move according to their timeline delta, so the pause of player does not stop the world, and there is no need to change anything else. Accumulation of the time, rather than calculation of $(\text{now} - \text{start} - \text{paused}) / \text{tic}$, guarantees that the scale changes have effect on the time after that moment, making entities not teleport, and unpaused entity continue to move from where it was paused. Child timeline always locks itself before its anchor, preventing deadlocks of threads on timelines.

(E) Section 2 - Client-Server

NetMessage is a compact 29-byte structure (type, client ID, tic, location, velocity) without SDL types such that it is usable by the headless server that never initializes a window. The server controls the dynamic platform ($x = 520 + 180 \sin t$) and assigns each client a unique thread and a 0MQ REP socket while using a mutex to protect the shared world state. There are at least three clients that act as separate processes and can come online whenever they like; the departure of one client does not halt the rest. NetworkClient listens on its own thread, and the game uses the most recent snapshot every frame.

(F) Section 3 - Multithreaded Game Loop

In accordance with the ThreadExample template (mutex, condition variable, busy flag), Engine::run generates two continuous threads:

- The player thread: advances PlayerTime and updates the entities on that thread.
- The world thread: advances WorldTime and updates the world entities and world callbacks, which execute network snapshots.

The main thread, in the meantime, renders finishedFrame, which is a duplicate of the last completed frame. The only thread that invokes SDL is the main thread. There is a stateMutex for each entity, and remote players are generated on the main thread and not in the workers' threads.

(G) Section 4 - Asynchronicity

Engine::paceFrame will pace the loop of each client according to the PlayerTime frequency (0.5 corresponds to 30Hz, 1.0 corresponds to 60Hz, 2.0 corresponds to 120Hz), and each client sends an update once per frame; therefore, the clients communicate with the server at different frequencies. The player is synchronized with the timeline, and the platform is synchronized with the server on WorldTime at 1.0, meaning that the platform exists in the same location on all screens. Each client has a separate thread in the server, which means that slowing one client does not slow down the rest.

(H) Section 5 - Peer-to-Peer

We also implemented option-2. Using the --p2p command, there are no servers running and no peer owns anything. Peers pass system_clock start values to each other, keep the minimum value as a common anchor point, and each calculates the platform based on the identical formula used by the server to make sure that there is an agreement among peers without any peer owning it. Peer with higher port number calls the peer with lower port number, thus providing only one connection between a pair of peers on their thread separately.

(I) Thread Safety and Concurrency

The engine separates the simulation from the rendering process. Simulation of the player and world takes place in persistent worker threads, while SDL input and rendering is done on the

main thread. State of the entities is protected using per-entity mutexes and state of the server by the server mutex. The remote players are spawned on the main thread, which ensures that no changes take place in the entity set while the workers are iterating over it.

(J) Individual Games

- Cave Ninja (Sai): ninja in PlayerTime, moving platform and wisp in WorldTime. Offline, client-server and peer-to-peer versions; the ninja uses the moving platform.
- Lost Under the Sea (Adit): diver in PlayerTime, fish in WorldTime. Offline, client-server and peer-to-peer versions, climbing, health, and rescue objectives.
- Hello Kitty Adventure (Yeva): Hello Kitty in PlayerTime; Kuromi and other players in WorldTime. Offline and client-server versions.

(K) Testing and Verification

We conducted our tests using multiple clients and peers concurrently:

- Three clients using one server where the movements were synchronized for all players and platforms.
- Stopping and changing one player's time line without interfering with any other clients.
- Clients running at 0.5x, 1x and 2x (30, 60 and 120 fps) watching the same platform.
- New clients connecting to a live server while one client disconnects and the others remain alive.
- Three peers without the server and then one of them disconnecting and the other two remaining alive.

(L) Build and Run

Make using `cmake -S . -B build` and `cmake --build build`; binaries can be found in `build/Release/`. All the servers, clients (`--client host port`), peers (`--p2p port peers...`), and time keys are listed in the README.